

PrismatAI Training Plan — V3-R2

Date: 2026-03-09 **Status:** FINAL — reviewed (9 external reviews + 2 meta-reviews), ready to implement

Approach: Track A — Supervised learning on human replays first; iterative self-play in a future phase **Author:** Surfinite + Claude Code

Key Verification Items

These are facts the plan depends on. If any are wrong, assumptions about data scale, feasibility, or success criteria may need revision.

- **V1 — Usable human replay count (validated, final):** Full balance validation (cost + rarity against current `cardLibrary.json`) completed across **all 203,602 codes** on March 7, 2026. Results: **154,061 balance-passed**, 48,707 failed, 834 fetch errors (S3 404s). After applying the full quality filter (both players ≥ 1500 rating at game time, rating change $\neq 0$ to exclude friendly/unrated games): **102,697 usable training games**. 740 unique players contribute. Post-patch pass rate: $\sim 97.8\%$. Pre-patch pass rate: $\sim 57.7\%$ — lower because many pre-patch games used unit costs/rarities that have since changed. This is the authoritative training set size.
- **V2 — Competitive unit count:** The authoritative unit list is `bin/asset/config/valid_units.json` and contains exactly **116 units**: 11 base set units (present in every game) and 105 dominion/random set units (the ranked competitive pool). This list was derived from the live game's unit selection screen, March 2026. The `cardLibrary.json` contains 161 entries but 45 of those are deprecated, event-mode, or removed units that no longer appear in competitive play. Any feature schema must be built on the **116-unit list**, not the raw `cardLibrary` count.
- **V3 — Balance validation definition:** A replay is considered balance-valid if its unit costs **and unit rarities (supply limits)** match the current `cardLibrary.json`. The validation script (`validate_db_codes.py`) has been extended to check both `buyCost` and `rarity` for every unit in every replay — it is fully general and catches changes from any era of the game's history.

Decision: validate all replays at training time. The January 14, 2019 cutoff was previously used as a fast filter (post-patch games assumed valid without checking). This approach is fragile — it relies on the boundary being correct and complete, and does not catch rarity/supply changes that happened at other points. Instead, every replay in the training set must pass the full cost+rarity validation before use, regardless of timestamp.

Rarity audit results (March 2026): A sample of 6,308 replays across 133 monthly buckets identified **17 units with historical rarity changes**:

- **Bombarder:** normal- $\rightarrow$ rare (supply 20- $\rightarrow$ 4), affected replays dated July 2018-March 2019. Approximately 544 training-eligible replays will be flagged.
- **16 other units:** mostly trinket- $\rightarrow$ normal or trinket- $\rightarrow$ legendary transitions, clustered around November 2015. These predate most high-rated play and will affect a smaller number of training-eligible replays.

Rarity mismatches are reported as `issue: "rarity_changed"` alongside the existing `issue: "cost_changed"` failures. Both cause a replay to fail validation and be excluded from the training set. Audit data: `c:\libraries\prismata-replay-parser\rarity_audit_results.json`.

- **V4 — JS engine accuracy:** The JavaScript engine is a transpilation of the original AS3 engine and is considered authoritative. Previously validated against replay data (100% pass rate on 500 replays), but **revalidation required** — the JS engine code was modified during recent integration work. Run a fresh 500-replay validation before Phase 2 extraction begins. During Phase 2a extraction, every replay will be validated as it passes through the engine. Any failures will be logged and excluded.
- **V5 — MasterBot as primary benchmark:** The target for supervised training is: the trained model, used as the evaluation function in Alpha-Beta HPS search, beats MasterBot (the Steam client's native AI). MasterBot is the strongest known Prismata AI and the most meaningful real-world reference. Access is via **SteamAI** (`js_engine/steam_ai.js`), which spawns Steam's native `PrismataAI.exe` per turn and communicates via stdin/stdout JSON. Fully integrated into the matchup runner: `node matchup_clean.js --player SteamAI --games N`. Supports parallel workers and player-switch mode.
- **V6 — Dataset composition, record scale, and supplementary data:**
 - **PvP status confirmed:** The 102,697 usable training games are all PvP — already verified. MasterBot has a rating ~ 1.0 and is excluded by the `rating >= 1500` filter, so no human-vs-bot games appear in the primary training set.
 - **Record count estimate:** At approximately 30 turns average game length, with **one record per player-turn (start-of-turn states only)**, the 102,697 games yield roughly **3.1 million raw training records** ($102,697 \text{ games} \times \sim 30 \text{ player-turns per game} \approx 3.1\text{M raw}$). Mid-turn states (after individual actions within a turn) are explicitly excluded — they introduce distribution shift, label ambiguity, and feature complexity without established benefit. **Codebase confirmation:** HPS search evaluates only end-of-turn states. `MoveIterator_PPPPortfolio.cpp` completes all 4 `PartialPlayer` phases (Defense, ActionAbility, ActionBuy, Breach) before returning child states to the search. The evaluator is never called on mid-turn positions. Start-of-turn training data matches the search's evaluation context exactly. VERIFY actual average game length from a sample of balance-validated replays; the 30-turn estimate is unconfirmed and the record count scales directly with it.
 - **"Grey data" — human vs MasterBot replays:** A potentially large pool of human-vs-MasterBot games may exist in the replay database. These are lower-confidence than PvP data: the human player is typically stronger than MasterBot (so the win-probability target is informative), but strategic intent is unknown — the human may have been experimenting, testing unusual strategies, or playing casually. This data could supplement training if the primary 103K dataset proves insufficient, but should not be mixed into the primary training set without a separate quality gate. Assess the size of this pool from the replay database and note as a contingency data source.
- **V7 — Rating distribution (measured, confirmed):** The usable training dataset (102,697 games, both players ≥ 1500 , rated) has a mean combined rating of approximately **3,909** (per-player average $\sim 1,954$). Distribution: 2000+ = 38.9%, 1800-1999 = 47.2%, 1600-1799 = 13.6%, 1500-1599 = 0.2%. Only ~ 230 games fall in the 1500-1599 bracket. This is strong news: the imitation ceiling concern is largely

addressed. The dataset is dominated by high-level play, not diluted by low-rated games. The 1500 floor adds almost no weak games. These figures are stable.

- **V8 — Unit properties available for set-based encoding:** All 105 random-pool units have structured properties in `cardLibrary.js`: buy cost (broken down by resource type), toughness, build time, default blocking, lifespan, fragile flag, and other numeric stats. These are encodeable as a fixed-length property vector per unit. Unit *abilities*, however, are defined as scripts rather than numeric values — encoding them requires either simplification (e.g. "does this unit generate attack? how much?") or omission. This is relevant to the set-based encoder approach described in Phase 1a Option 4.
- **V9 — C++ AI <-> JS engine click protocol correctness (hard gate for Phase 5 and Phase 6):** V4 verifies JS engine rule correctness; V9 verifies that the C++ AI's chosen moves translate correctly through the click protocol. Failures manifest as `skippedBuys` (clicks legal in C++ but rejected by JS) or incorrect game state after ability activation. All move types must work: BUY, USE_ABILITY (including shift-click), UNDO_USE_ABILITY, ASSIGN_BLOCKER, SNIPE/CHILL (two-step targeting), END_PHASE.

Verification protocol: Run a 512+ game LiveHardestAI vs MasterBot matchup (Phase 0b). Before Phase 5 begins, confirm:

- Zero `skippedBuys` logged across all games
- Zero click failures for all move types
- Game outcomes are plausible under human expert review

Phase 5 and Phase 6 remain hard-gated on this verification completing cleanly.

V9 failure triage: If `skippedBuys` or click failures are found, triage by move type:

- **BUY failures:** Check `Benchmarks.cpp` click emission (deck index mapping) and JS `Card.js` buy validation
- **USE_ABILITY / shift-click failures:** Check C++ multi-card ability expansion vs JS single-click handling
- **SNIPE/CHILL failures:** Check two-step targeting (C++ emits source+target, but source may already be clicked)
- **ASSIGN_BLOCKER failures:** Check inert-card blocking rules (`Card.cpp:canBlock()` vs AS3 `couldDefendThisTurn`)
- Fix in the layer where the bug originates (C++ move→click translation or JS click→action interpretation). Do not patch both sides to mask a single-side bug.

Overview

The goal is to train a neural network evaluation function for Prismata that, when used inside the existing Alpha-Beta Hierarchical Portfolio Search (HPS) engine, produces an AI stronger than MasterBot (the Steam client's native AI).

Churchill (2019) demonstrated this approach works — training on 500K AI self-play games produced an evaluator that beat both the resource heuristic and the playout evaluator. We are building on that result with three key differences:

1. **Accurate engine:** The JS engine correctly implements all game rules. Prior C++ approximations had known errors.
2. **Human replay data:** Human expert games carry stronger strategic signal than AI self-play (the AI being trained against will be stronger than prior AI self-play partners were).
3. **Architecture exploration:** Churchill's 2-layer, 512-neuron MLP was chosen without exhaustive search. We will run targeted ablations before committing to a full training run.

Self-play iteration (AlphaZero-style) is planned as a future phase. This plan covers supervised training only.

Known limitation — distribution shift: A model trained on expert human game positions will be evaluated on positions generated by AI search — which may include states humans rarely reach (deep tactical traps, unusual defensive formations, sacrifice lines). This is a fundamental limitation of supervised learning for game evaluation functions and is the primary motivation for Phase 6 (self-play iteration). Phase 5 evaluation should include qualitative review of cases where the model's evaluation appears confident but strategically wrong.

A note worth highlighting: *"The Stratego paper (Sokota et al. 2025) is probably the closest analog to what you're attempting in terms of game complexity and budget constraints — their approach of self-play RL + test-time search on an accessible budget maps almost directly to Prismata."* We agree, and it is the primary reference for the self-play phase (Phase 6).

A Note on Code Cleanup

The `cardLibrary.json` contains 161 entries, of which 45 are deprecated, event-mode, or non-competitive units. These units never appear in balance-validated competitive replays and should not appear in any training feature vector.

Recommendation: Before finalising the feature schema, prune `cardLibrary.json` to remove the 45 non-competitive entries. This is not strictly required (the `valid_units.json` whitelist already filters them out at runtime), but it reduces confusion, eliminates a source of latent bugs, and produces cleaner code going forward. This can be done as a standalone task independently of the training pipeline.

Phase 0: Prerequisites and Baseline Measurements

Goal: Complete all setup before any training work begins. Nothing in Phase 1+ should start until all items here are done and recorded.

0a. Complete Balance Validation (DONE)

- Full re-validation of all 203,602 codes completed March 7, 2026 — single clean run with current cost+rarity validator.
- Results: 154,061 balance-passed, 48,707 failed, 834 S3 fetch errors.
- Results imported into `replays.db`. Final usable training set: **102,697 games** (both players ≥ 1500 , rated, balance-passed, all eras).
- Produce final canonical code list: `balance_validated_1500plus.json` — pending export from DB.

0b. Benchmark Tournament (TODO)

- Run 512+ game **LiveHardestAI vs MasterBot** matchup via SteamAI (`node matchup_clean.js --player-white LiveHardestAI --player-black SteamAI --games 512`). Record win rate.
- This run also serves as V9 click-protocol verification — confirm zero `skippedBuys` and zero click failures.
- **SteamAI determinism check:** Run the same game state through MasterBot multiple times (at least 10 identical positions from different game phases). We believe MasterBot is non-deterministic, but this should be confirmed. If it is deterministic, paired matches produce correlated results that inflate confidence intervals — statistical analysis must account for this.
- Human expert visual review of a sample of games should confirm strategic coherence.
- MasterBot is the primary evaluation target for Phase 5.

0c. JS Engine Validation on Representative Sample

- Run replay validation on a fresh random sample of 1,000 balance-validated codes as a pre-extraction sanity check.
- Record pass rate.
- **Decision gate:** If pass rate is below 95%, investigate failures before proceeding. Systematic failures indicate an engine correctness issue that would corrupt training data.
- **Full validation during extraction:** During Phase 2a, every replay will be validated as it passes through the engine. Any replay that fails JS engine stepping will be logged and excluded. This makes the sample check a fast pre-flight, not the sole validation.

0d. Dataset Composition Audit

- For the final code list: measure PvP vs PvAI ratio (V6)
- Measure rating distribution (how many 1500-1799, 1800-1999, 2000+)
- Measure game length distribution (average turns per game for record count estimate)
- Measure card set distribution (base-only vs random-set games)
- **Measure draw rate** across the training set. If draws exceed 2% of games, consider separate handling (e.g., label = 0.5 rather than forcing a win/loss assignment).
- **Measure first-player win rate** across the training set. If P1 or P2 win rate deviates from 50% by more than 2 percentage points, record the asymmetry for use in the Elo prior formula (Phase 1b).
- **Identify forfeit/timeout/resignation games.** Check whether the replay data distinguishes between wins by destruction (all opponent units destroyed), wins by resignation, and wins by timeout. If timeouts are common (>1% of games), exclude them. **Resignation label noise mitigation (default: trim).** Players resign when they *believe* they're losing, which may be 5-10 turns before actual loss. The last 3-5 positions before a resignation carry label 0 (loss) but the true win probability may be 0.15-0.30. **If >5% of games end by resignation: trim the last 3-5 pre-resignation positions from the training set by default.** Run one ablation *without* trimming to quantify the effect. This is the stronger default because resignation noise is systematic (biased toward labeling close games as decisive) and correlated with player identity (aggressive resigners create different noise than stubborn players).
- **Measure per-unit frequency distribution** across all 102,697 games. Note any units appearing in fewer than ~500 games as "low-data units" — evaluation quality for sets containing these units will be poor. Flag these units for separate reporting in Phase 5a.
- **Turn-bucketed label entropy:** Report label entropy by turn bucket (e.g., turns 1-8, 9-18, 19+) to quantify noise level at different game phases. This informs the label strategy decision in Phase 3.
- Record all findings. They directly inform feature schema choices and training split decisions.

0e. C++ Inference Speed Benchmark (Before Architecture Commitment)

Before Phase 3 architecture decisions are finalised, benchmark how quickly the C++ `NeuralNet.cpp` inference engine evaluates positions at different model sizes. This trade-off — model quality vs. search depth — is the central tension and should be measured, not assumed.

- Benchmark: evaluate 10,000 positions with models of sizes 128h/2L, 256h/2L, 256h/3L, 512h/2L, 1024h/2L. **Also benchmark residual MLP variants:** 256h/4R, 128h/4R (4 residual blocks). Record milliseconds per evaluation.
- ****Benchmark in the actual search loop at realistic think times**** (3s and 7s): measure ****nodes searched**** (not just raw evals/sec) with each model size loaded into a real HPS game loop on representative card sets. Raw batch benchmarks overestimate deployment speed due to cache effects, branch-heavy code around evaluation, and irregular call patterns. Record: (a) raw evals/sec, (b) nodes searched at 3s, (c) nodes searched at 7s, (d) effective search depth achieved. Also measure nodes searched with a ****constant evaluator**** (instant return) to establish the search budget ceiling — this reveals HPS's effective branching factor.
- If a 1024h model takes 0.5ms but a 256h model takes 0.05ms, the smaller model gets 10x more search depth at fixed think time. At 7 seconds think time, this is likely worth more than the quality gain from a larger model.
- Record results and use them as a constraint on Phase 3 architecture selection. ****Derive the minimum eval speed**** from the measured search budget: ``floor = search_budget_at_target_think_time / think_time_seconds``. The previous heuristic of 500 evals/sec was a guess — the actual floor depends on HPS's branching factor, which varies by card set and game phase.

0f. Optional: cardLibrary Cleanup

- Identify and remove the 45 non-competitive unit entries from `cardLibrary.json`
- Update any code that references them (JS engine, C++ loader)
- This is independent of the training pipeline; can be parallelised with other Phase 0 work

Phase 1: Feature Schema Design

Goal: Define a canonical, versioned feature vector before any data extraction begins. This schema must be agreed upon and frozen before Phase 2 starts. Changing the schema after extraction requires full re-extraction.

1a. The Unit Representation Problem

A game of Prismata uses 11 base set units plus 8 randomly selected units from the 105-unit dominion pool — so at most 19 unit types are present in any given game. However, different games use different random units. For the feature vector to be universal (usable across all card sets), it must accommodate all 116 possible unit types, with zero-counts for types not present in the current game.

Four representation options were considered:

Option 1 — Type-Count (Churchill 2019 Baseline)

For each of the 116 unit types, record how many units of that type each player owns. Add resource counts and the current player indicator.

- **Feature vector size:** 116×2 (count per player) + 12 (resources: 6 types $\times$ 2 players) + 1 (player to move) = **~245 features**
- **Advantages:** Simple, fast inference, established precedent. Churchill achieved competitive results with this approach.
- **Disadvantages:** Loses all instance-level information: which units are under construction, which have been used this turn, which are currently blocking, which have taken partial damage.
- **Churchill's note:** "This encoding discards information such as which units may be activated, individual unit instance properties... since the states are all recorded at the beginning of each turn when units are not yet activated, much of the effect of this information loss is alleviated."

Option 2 — Type-Count with Instance Flags, Supply, and Card-Set Indicator (Selected)

Extend Option 1 with per-type aggregate flags, supply tracking, and a card-set presence indicator. This matches the feature layout already implemented in `NeuralNet.cpp`.

For each of the 116 unit types, encode **11 values**:

- P0 ready count, P0 constructing count, P0 exhausted/ability-used count, P0 blocking count
- P1 ready count, P1 constructing count, P1 exhausted/ability-used count, P1 blocking count
- P0 supply remaining, P1 supply remaining
- In card set (1.0 if this unit type is available for purchase in this game, 0.0 otherwise)

Plus **14 global features**:

- P0 resources (gold, blue, red, green, energy, attack) — 6 values
- P1 resources (gold, blue, red, green, energy, attack) — 6 values
- Turn number (normalized)
- Active player indicator (0 or 1)
- **Feature vector size:** $116 \times 11 + 14 = \mathbf{1,290 \text{ features}}$
- **Advantages:** Captures tactically relevant state including construction status, ability usage, blocking assignments, supply exhaustion, and card-set composition. The `in_card_set` flag lets the model distinguish "not in this game" from "in this game but currently unbought." Supply remaining is strategically critical — knowing the opponent has bought 3 of 4 available Tarsiers is qualitatively different from knowing they've bought 3 of 20.
- **Disadvantages:** ~5x larger than Option 1; more expensive to extract and slower to infer (though still well within C++ inference speed requirements at this scale).
- **Why this is selected:** The JS engine reliably exposes all these per-instance fields. Using them is the primary advantage of the accurate JS engine over the prior C++ approximation. **The C++ inference engine already implements this exact layout** (`NeuralNet.cpp:266-388`) — no C++ changes needed. Run Option 1 as a Phase 3a baseline for comparison.

Option 3 — Per-Instance Encoding — Maximum expressiveness; variable-length input. Future experiment if Options 1/2 plateau.

Option 4 — Set-Based / Property Encoding — Encode unit *properties* rather than identity. Generalises across card sets. Promising medium-term direction; requires ability schema design. See V8.

1b. Label Design

- **Training target:** probability that **Player 0 (P0) wins** the game from this state. P0 is always the player whose features appear first in the feature vector (game index 0 = Player_One in C++, `inst.owner === 0` in JS). The active player indicator is an input feature (0 if P0 is active, 1 if P1 is active), NOT a selector for which player's win probability to predict. This convention is confirmed consistent across C++ (`NeuralNet.cpp:283-285`), JS (`state_adapter.js:186`), and Python (`vectorize.py`). **Document in `schema_v1.json`: "target = P(Player_0_wins)".**
- **Raw label:** 1 if P0 won, 0 if P0 lost (draw = 0.5)
- **Canonical loss/output/inference path (Path A — logit/probability-native):**
 1. **Training:** Model outputs a raw scalar logit z . Loss = `BCEWithLogitsLoss(z, y)` where $y \in [0, 1]$ is the probability label.
 2. **Python validation:** Convert with $p = \text{sigmoid}(z)$ for probability interpretation.
 3. **C++ export:** Apply $v = 2 * \text{sigmoid}(z) - 1$ to produce the $[-1, 1]$ signed value that `Eval.cpp` expects, then scale by $\times 100$ (pure NeuralNet mode) or $\times 10000$ (NeuralNetPlusLayout blend mode).
 4. **Implementation note:** The existing C++ `NeuralNet.cpp` applies `tanhf()` on the raw output. For Path A compatibility, replace `tanhf(rawValue)` with `2.0f * sigmoidf(rawValue) - 1.0f` — mathematically equivalent if training uses the matching loss. Alternatively, train with tanh output + MSE on $[-1, 1]$ targets (Path B). **Path A is recommended** because BCE is the proper scoring rule for probability estimation and handles soft labels (Strategy C/D) more naturally.
- **Loss function:** Binary cross-entropy (BCE) on value prediction. BCE is the theoretically correct proper scoring rule for probability estimation and handles soft labels in $[0.3, 0.7]$ more naturally than MSE. Churchill found MSE and BCE produce similar results with hard labels, but with soft labels BCE is the principled default. **Run MSE and Huber loss as Phase 3 ablations** to verify. Huber caps gradient for outlier labels (e.g., player resigns a winning position — label 0 for an objectively strong state), reducing sensitivity to mislabeled games.
- **Implementation:** Use `torch.nn.BCEWithLogitsLoss` (not `BCELoss` after manual sigmoid). The `WithLogits` variant is numerically stable for soft targets and avoids $\log(0)$ when predictions are near 0 or 1.
- **Primary offline metric:** BCE loss on the temporal validation set is the primary metric for comparing ablation runs. Report Brier score as a secondary diagnostic.

The Noisy Label Problem (Critical)

Binary game outcomes are a very noisy signal for early-turn positions. A turn-1 position from a game Player 1 eventually won gets label 1.0, but the true win probability at turn 1 is close to 0.5 for both players. The model

must average out this noise, and it is working from a very poor signal-to-noise ratio in the opening and midgame — precisely where evaluation quality matters most for search.

This problem is more acute for high-rated human games than for Churchill's AI self-play. AI self-play games are shorter and more decisive (errors compound faster at lower skill). Human 2000-rated games are longer and closer, meaning the true win probability drifts slowly from 0.5, making the binary label noisier for more turns.

Label strategy: 3-way ablation (Phase 3)

The following four label strategies will be compared in Phase 3 ablations. Do not commit to one before empirical results:

Strategy A — Hard binary labels (baseline) Use raw 0/1 labels. Simplest, no assumptions. The model must average out all noise itself.

Strategy B — Hard labels with temporal sample weighting Keep 0/1 labels but weight each sample's loss contribution by game progress:

```
REFERENCE_LENGTH = 40 # approximate 75th percentile game length
t = min(1.0, ply_index / REFERENCE_LENGTH)
sample_weight = 0.3 + 0.7 * t # early positions contribute less to loss
```

This downweights noisy early positions without modifying the labels themselves. Lower ceiling than Strategy C but requires zero label schema changes.

Strategy C — Elo-interpolated temporal labels Blend the game outcome with an Elo-based prior that decays over the course of the game:

```
elo_prior = 1 / (1 + 10 ** ((opponent_rating - player_rating) / 400))
REFERENCE_LENGTH = 40 # fixed constant, NOT total_turns
t = min(1.0, ply_index / REFERENCE_LENGTH)
label = (1 - t) * elo_prior + t * actual_outcome
```

The interpolation parameter `t` uses a **fixed reference length** (approximately the 75th percentile game length) rather than `total_turns`. This eliminates outcome-correlated information leakage — using `total_turns` would encode game length, which is correlated with how decisive the game was (short games = stomps, long games = close).

Implementation note: If Phase 0d reveals a first-player advantage >2pp, adjust the Elo prior for equal-rated games accordingly (e.g., if P2 wins 53%, use `elo_prior = 0.53` for P2 at equal ratings instead of 0.50).

Non-linear ramp variant (sub-ablation within Strategy C): The first 10 turns of a Prismata game reveal less about the winner than the last 10. A non-linear ramp concentrates label authority in the late game:

```
# Quadratic ramp – early labels stay close to Elo prior longer
t_raw = min(1.0, ply_index / REFERENCE_LENGTH)
```

```
t = t_raw ** 2 # quadratic: slow start, fast finish
label = (1 - t) * elo_prior + t * actual_outcome
```

Test quadratic (t^{**2}) alongside linear in the Phase 3 label ablation. If the difference is negligible, use linear (simpler).

Strategy D — Neutral 0.5-prior temporal labels A simpler alternative to Strategy C that avoids injecting player-skill information into the target. The deployed evaluator won't know player ratings, so Elo-conditioned labels train a target the model cannot reproduce at inference time:

```
REFERENCE_LENGTH = 40 # fixed constant, NOT total_turns
t = min(1.0, ply_index / REFERENCE_LENGTH)
label = (1 - t) * 0.5 + t * actual_outcome
```

This directly addresses early-game label noise without the Elo dependency. If Strategy C and D produce similar results, prefer D (simpler, no external data dependency).

Note on constants: The turn normalization cap in the feature schema (/50) and the label interpolation `REFERENCE_LENGTH` (40) are intentionally different constants. The former normalizes the `turn_number` input feature; the latter controls label blending rate using `ply_index`. Do not "fix" them to match — they serve different purposes.

- *TD(lambda) bootstrapping* (higher ceiling, more complex): use an initial model's own predictions to generate soft labels for a second pass. Standard in game AI but requires an iterative training loop. Consider for self-play iteration 2+ (Phase 6).

1c. Schema Versioning

- Schema stored as a single versioned JSON file: `training/schema_v1.json`
- Must include:
 - **Player convention:** "P0 = Player_One (game index 0, white). P1 = Player_Two (game index 1, black). Active player stored as input feature, NOT used to rotate P0/P1 slots." This is absolute player numbering, confirmed consistent across C++ (NeuralNet.cpp:283-285), JS (state_adapter.js:186), and Python (vectorize.py).
 - **Value target:** "target = P(Player_0_wins)" — probability that P0 wins, regardless of whose turn it is.
 - Unit list (ordered)
 - Feature names and indices
 - Value ranges
 - **Normalization method and constants** — the C++ inference engine uses `clamp_divide` normalization with specific per-feature caps. Initial values derived from WillScore resource values (Attack=2.25, Blue=1.50, Green=1.20, Gold=1.00, Red=0.90, Energy=0.50) and typical gameplay accumulation patterns:
 - Gold: /25 (10-25 Drones common in mid-late game)
 - Blue: /4 (1-3 Blastforge + misc = 1-4, some units add more)
 - Red: /8 (1-3 Animus × 2 red + misc = 1-8)

- Green: /16 (1-10 Conduits × 1 green + misc producers and accounts for accumulation)
 - Energy: /8 (typically 0-4, sometimes pushed higher)
 - Attack: /30
 - Turn: /50 (games occasionally exceed 30 turns)
 - Unit counts: raw (no normalization — typically 0-20)
 - Supply remaining: raw
 - Binary flags (in_card_set): 0 or 1
 - These are **initial estimates** — Phase 0d dataset audit will compute actual percentile distributions and adjust caps so <1% of values are clipped
 - The JS extraction pipeline must apply **identical normalization constants**. Mismatches produce silently wrong evaluations in C++.
 - Label encoding (which strategy from Phase 1b was selected)
 - Any change to the feature vector = new version. Old data cannot be mixed with new schema.
 - The canonical unit ordering comes from `valid_units.json` (base set first, then random set alphabetically, or by some consistent ordering). This ordering must be fixed and documented.
 - **Include a schema hash/checksum in output filenames and binary shard headers** to prevent accidental mixing of schema versions. Filenames can be renamed; headers can't.
-

Phase 2: Data Extraction Pipeline

Goal: Extract all usable human replay data through the accurate JS engine into a single canonical dataset. All previous JSONL files are considered legacy and are not mixed into this output.

2a. Extraction

- **Input:** `balance_validated_1500plus.json` from Phase 0a
- **Process:** For each replay code, fetch from S3, step through game using JS engine, record **one training record per player-turn at start-of-turn states only**. Mid-turn states (after individual actions within a turn) are excluded — they introduce policy-dependent distribution shift, label ambiguity, and feature complexity.
- **Output:** Binary shard format (matching the existing selfplay pipeline structure for loader compatibility), or HDF5 with chunked datasets. JSONL at 3.1M records (~15-29 GB) is impractically slow to load.
- **Record metadata:** Each record must include: `replay_code`, `turn_number`, `rating_p0`, `rating_p1`, `game_date`, `card_set` (the 8 random unit IDs present in this game). These enable temporal splitting, stratified analysis, and post-hoc filtering without re-parsing replays.
- **Include `total_turns`** in each record: required to compute Elo-interpolated labels (Phase 1b Strategy C) at training time without re-parsing replays. Note: `total_turns` is used only for label computation and is **never** an input feature to the model. **Hard requirement:** Store metadata (`total_turns`, `rating_p0`, `rating_p1`, `game_date`, `replay_code`) in a separate metadata block from the feature vector. Add an extraction test that verifies no metadata field appears in the feature vector indices.
- **Note:** Store raw state fields alongside the feature vector in each record if feasible. This allows re-vectorisation (re-running the featurisation step without re-fetching replays) if the schema changes during Phase 3 ablations.
- **Engine validation during extraction:** Validate every replay as it passes through the JS engine. Log any replays that fail engine stepping (invalid moves, state inconsistencies). If the failure rate exceeds 1%,

halt and investigate before proceeding. This makes V4 verification comprehensive rather than sample-based.

- **Parallelization:** Each replay is independent. Use 4-8 worker processes to reduce extraction time from ~14 hours to ~2-4 hours.

Symmetry augmentation — NOT applied by default:

Prismata has symmetric *rules* but **asymmetric starting positions**: P0 begins with 6 Drones, P1 begins with 7 Drones (compensation for going second). This means naive player-swap augmentation (swap P0/P1 features, invert label) creates unreachable game states in the early game. A mirrored turn-1 state puts P0 at 7 Drones — a position that never occurs in any real game. Training on impossible states introduces noise exactly where the noisy-label problem is already worst.

By mid-game (approximately turn 8-10), both players have purchased enough additional Drones that the 1-Drone starting difference is <10% of total economy. Mirrored states become approximately plausible. By late game the asymmetry is negligible.

Default: Do not apply mirror augmentation during extraction. The dataset already contains records from both P0's and P1's turns in every game, providing natural perspective diversity without synthetic augmentation.

Phase 3 ablation (see 3c-3): Test turn-gated mirroring as a data augmentation strategy:

1. No mirroring (baseline)
2. Turn-gated mirroring (only positions after turn N, where N is set so the starting asymmetry is <10% of mean economy — approximately turn 8-10, calibrated from Phase 0d data)
3. Full mirroring (all turns — to quantify the damage from early-game noise)

If turn-gated mirroring improves validation loss and arena performance, adopt it for Phase 4. If no mirroring ties or wins, the simpler approach is preferred.

Schema requirement (preserved): Design the Phase 1 schema with P0 features in a contiguous block followed by P1 features in the same layout, so that mirror augmentation (if adopted after ablation) is a single slice-and-swap operation.

Metadata: Store `ply_index` (half-turn index from game start, incrementing each player-turn) alongside `turn_number` in each record. `ply_index` more precisely captures tempo — when it's P0's turn, P0 is "half a turn ahead" in tempo, and vice versa. Use `ply_index` for label computation (temporal weighting, soft labels) rather than `turn_number`.

Base-only game exclusion:

Base-only games (no random units — both players use only the 11 base-set cards) are strategically very different from the random-set format that constitutes all competitive play. Including them introduces a distribution the model must learn that has near-zero relevance to the target deployment context. Exclude base-only games from the primary training set. Measure their count during Phase 0d so the exclusion can be quantified.

2b. Train / Validation / Test Split

- **Primary split: temporal.** Sort all replays by game date. The most recent 10% of games = test set, the next most recent 10% = validation set, the remaining 80% = training set. This better reflects

deployment conditions: the model will face positions generated by current-era play, not a random sample of historical play. Prismata's strategic meta evolved over time — opening strategies, unit valuations, and player styles have shifted.

- Within each temporal partition, no further splitting is needed — replay-code-level integrity is maintained by the temporal ordering.
- **Secondary diagnostic: random split.** Also compute validation metrics on a random 10% holdout to compare against the temporal split. If temporal validation loss is much worse than random, the model is memorizing historical patterns rather than learning general strategy.
- Test set held out entirely — not used during Phase 3 architecture search.
- ****If turn-gated mirroring is adopted after Phase 3 ablation**, apply it AFTER the split.** A mirror record from a test game must never appear in training. Assign each replay code to a split first, then generate mirror records within that split.

2c. Deduplication

- Some codes may appear across multiple source lists. Deduplicate by code before extraction begins.
- After extraction, verify no duplicate records (same replay_code + turn combination)

2d. Post-Extraction Validation

- Spot-check 100 random records against expected game outcomes
- Verify label distribution (~50/50 expected given symmetrically-rated games)
- Verify feature value ranges (no NaN, no Inf, no obviously wrong counts)
- Verify total record count is consistent with game length estimate
- **Mirror augmentation correctness test (run only if adopting turn-gated mirroring from Phase 3 ablation):** For a sample of 1,000 records above the turn threshold, verify:
 - `mirror(mirror(record)) == record` (double-mirroring recovers the original feature vector exactly)
 - `label_original + label_mirror == 1.0 exactly` (labels invert correctly) — this must hold for all four label strategies including Strategy C and D.
 - All player-specific features (unit counts, resources, supply) swap correctly between P0 and P1 blocks
 - The active player indicator flips
 - **Verify that mirrored records are above the turn-gate threshold** — no early-game states should be mirrored
- **Seat asymmetry audit (always run):** Report P0-active vs P1-active record counts, first-player win rate per split, and average game length in ply. Verify the starting asymmetry (P0: 6 drones, P1: 7 drones) is reflected in early-game feature statistics.
- **Three-way feature parity test (JS → Python → C++):** For 100 game states, extract features via: (a) JS engine (`state_adapter.js`), (b) Python vectorizer (`vectorize.py`), and (c) C++ inference engine (`NeuralNet.cpp:extractFeatures`). All three must produce the **exact same feature vector** (within float32 tolerance). This catches normalization mismatches, player-ordering bugs, and unit-index mapping errors across the pipeline.
- **Training data loader verification:** Load 100 records via the actual PyTorch training pipeline (DataLoader → batch → model input), decode them back to human-readable features (unit names,

resource values, turn number), and verify against the original extraction output. This catches serialization/deserialization bugs (shape mismatches, byte-order errors, off-by-one in feature indexing) that produce models that train fine on garbage but evaluate nonsensically.

- **Per-feature statistics:** Compute mean, std, min, max across the full dataset for every feature dimension. Flag any feature with zero variance (e.g., blocking count always 0) or implausible statistics (negative counts, values outside normalized range). This catches feature extraction bugs that are invisible in spot-checks but corrupt the entire dataset.

Phase 3: Architecture Search and Ablation

Goal: Determine a good architecture before a full training run. Use a representative subset of training data. Do not proceed to Phase 4 without reviewing results here.

This phase is the primary place where the plan should be adjusted based on empirical findings. **Track ablation runs with W&B or MLflow** — with 15+ configurations across architecture, features, labels, and regularization, spreadsheet tracking becomes error-prone.

3a. Churchill Baseline (Lower Bound)

Replicate Churchill (2019) as closely as possible on our data:

- Feature set: Option 1 (type-count, ~245 features)
- Architecture: 2 hidden layers, 512 neurons, tanh activation
- Optimiser: Adam, LR = $1e-5$
- Loss: MSE, value prediction only

This is the lower bound. If a more complex model cannot beat it, something is wrong with the model, the data, or the problem formulation.

3b. Feature Richness Ablation

Train same Churchill architecture on Option 2 features (~1,290 features) and compare validation loss.

- If Option 2 adds less than ~1 percentage point: determine whether the complexity is justified
- If Option 2 adds ≥ 2 percentage points: Option 2 is likely the better starting point

3c. Capacity Search and Architecture Variants

Vary hidden dimension: 128, 256, 512, 1024. Vary depth: 2, 3, 4 layers. Use the winning feature set from 3b.

Key question: is 512 neurons (Churchill's choice) actually optimal, or is it over/under-powered for a ~245 or ~1,290-feature input? A smaller model infers faster in C++ (directly increases search depth at fixed think time). A 256-neuron model may be preferable to a 512-neuron one if accuracy is similar. Cross-reference against the Phase 0e inference speed benchmark.

Residual MLP blocks: Include at least one run with skip connections (residual blocks) alongside the plain MLP:

```
class ResBlock(nn.Module):
    def __init__(self, dim):
        super().__init__()
        self.fc1 = nn.Linear(dim, dim)
        self.fc2 = nn.Linear(dim, dim)
        self.norm = nn.LayerNorm(dim)
    def forward(self, x):
        return x + self.norm(F.relu(self.fc2(F.relu(self.fc1(x)))))
```

Skip connections allow 4-6 block depth without degradation and are trivially portable to C++. **The C++ inference engine already supports residual blocks with LayerNorm** ([NeuralNet.cpp:146-154, 442-459](#)). They do not add significant inference overhead at these model sizes. Compare a 4-block residual MLP against the plain MLP winner from the depth/width search.

Ablation efficiency: Use a **fixed epoch count** (e.g., 30 epochs with early stopping) for architecture comparison. Training is a one-time cost; what matters is which architecture *learns best* (fixed epochs) and which architecture *infers fastest* (Phase 0e constraint). These are separate questions — wall-clock budgets conflate them. Apply the Phase 0e inference speed constraint as a **separate hard filter** after identifying the best-learning architectures. **Report both validation-loss-at-epoch-N and validation-loss-at-wall-clock-time-T** for all ablation runs. The primary comparison is fixed-epoch, but loss-at-time is free information that may be useful if revisiting the wall-clock vs fixed-epoch question later.

3c-2. Label Strategy Ablation

Compare the four label strategies from Phase 1b on the best architecture from 3c:

1. **Strategy A:** Hard binary labels (0/1)
2. **Strategy B:** Hard labels + temporal sample weighting
3. **Strategy C:** Elo-interpolated labels with fixed reference length
4. **Strategy D:** Neutral 0.5-prior temporal labels (no rating dependency)

Use the same training budget and hyperparameters for all four. Compare validation loss (and calibration if feasible). The winner becomes the label strategy for Phase 4. If C and D produce similar results, prefer D (simpler, no external data dependency, avoids training a target the model cannot reproduce at inference time).

3c-3. Sampling and Weighting Ablations

Long games contribute more records than short games, causing them to dominate gradient updates. A 50-turn game contributes ~2.5x more gradient than a 20-turn game, and within-game samples are highly correlated. This is not just a statistical footnote — it changes what the model learns.

Compare:

1. **Baseline:** Uniform sampling over all records (current default)
2. **Inverse-game-length weighting:** Weight each sample by `1 / total_turns` (equalizes per-game contribution)
3. **Per-game uniform sampling:** Sample uniformly by game, then uniformly by turn within game (per-epoch reshuffling)

4. **Capped per-game contribution:** Max N records per game per epoch

Additionally, test **rare-unit-aware sampling**: oversample games containing low-frequency units (flagged in Phase 0d) by 2-3x. This prevents the model from becoming blind to infrequent units that still appear in competitive play.

Mirror augmentation ablation: Test whether turn-gated player-swap augmentation improves results despite Prismata's seat asymmetry (P0: 6 Drones, P1: 7 Drones):

1. **No mirroring** (baseline — default for V1)
2. **Turn-gated mirroring** (only positions after turn N): swap P0/P1 features, invert label, flip active player indicator. Set N so the starting Drone asymmetry is <10% of mean total economy — approximately turn 8-10, calibrated from Phase 0d data. This yields ~1.5-1.7x augmentation.
3. **Full mirroring** (all turns — to quantify the damage from early-game impossible states)

If turn-gated mirroring improves validation loss AND mini-arena performance, adopt it for Phase 4. If no mirroring ties or wins, prefer the simpler approach.

Weight monitoring: When combining multiple weighting schemes (label strategy + rating weighting + temporal weighting + per-game weighting), log the effective weight distribution as a histogram. Verify that at least 80% of samples have effective weight > 0.3. If the tail is too thin, the optimizer is seeing a much smaller effective dataset than the raw record count suggests.

3d. Policy Head Experiment (Recommended Ablation)

Add a policy head predicting **buy counts per unit type** (not full-turn move imitation). A Prismata turn is a composite action (defend, activate abilities, buy, breach) where many action orderings are equivalent — predicting the full click sequence is noisy and underspecified. Buy-count prediction is well-defined: for each unit type, predict how many were purchased this turn. The C++ PUCT infrastructure already implements this: [UCTSearch.cpp:218-230](#) sums policy logits for each bought unit type in a candidate move, then softmaxes across moves to produce priors. The training pipeline uses MSE + 0.5×BCE hybrid on buy counts ([train.py:151-164](#)). Even weak policy accuracy (20-25%) can significantly improve Alpha-Beta search quality via PUCT-style move ordering — this is especially valuable in games with Prismata's extreme branching factor.

- If move-prediction accuracy reaches ~20%+, include the policy head in the Phase 4 full training run.
- If accuracy is below ~15%, the signal is too weak to help.

3e. Search-Position Sanity Check (Before Decision Gate)

Before the final architecture decision, verify that top candidates behave sanely on positions generated by AI search (not just human replay positions). This was previously only in Phase 5 — moving it here catches distribution shift before committing to a full training run.

- Run HPS + candidate eval on ~200 self-generated positions from search
- Check value consistency: positions that are objectively similar should have similar values
- Check value symmetry on mid/late-game positions: `value(state) + value(mirror(state)) ~= 1.0` (game rules are symmetric even though starting positions aren't — this should hold approximately for positions past the opening)
- Check tactical sanity: does the model correctly prefer obviously winning positions?

- If a candidate looks good on validation but produces erratic values on search-generated positions, this signals distribution shift — flag for investigation before proceeding.

This is a fast diagnostic (~10 minutes per candidate) that can save hours of wasted full-training runs.

3f. Architecture Decision Gate

Before Phase 4, review:

- Which feature set won (Option 1 vs 2)?
- What is the optimal depth and width?
- Does validation accuracy plateau before the full dataset? (If yes: data-limited, not capacity-limited — consider self-play data generation sooner)
- Is a policy head worth including?
- Which label strategy won (A, B, C, or D)?
- Which sampling/weighting strategy won?

Decision rule for ambiguous results: If the feature-richer model loses more than 0.1ms/eval vs the simpler model (from Phase 0e) but improves validation loss by less than 0.005, prefer the simpler model — inference speed dominates at fixed think time. If results are genuinely tied, prefer the simpler architecture.

Mini-arena validation (required for top 2-3 candidates): Every serious candidate must pass a cheap play test before full training. Run 64-128 paired games vs a fixed baseline (LiveHardestAI or MasterBot) at 3s think time. Use identical opening-book settings for all candidates. This is not for statistical significance — it's a sanity check that offline metrics correlate with actual playing strength. If a candidate with worse validation loss plays better, investigate before choosing.

3g. Feature Importance Analysis

After selecting the winning architecture, run **SHAP or permutation importance** on the trained Phase 3 model to identify which of the 1,290 features (Option 2) the model actually uses. If the model relies on ~50 features and ignores the rest, this informs whether the richer feature set is pulling its weight or just adding noise. This is a diagnostic — it does not block Phase 4, but may inform future feature simplification.

- Run on a sample of ~10,000 validation set positions
- Report top-50 most important features by name/category
- Report bottom-50 (candidates for removal if simplification is ever needed)
- If Option 1 (~245 features) won in 3b, skip this step — no feature bloat to diagnose

Phase 4: Full Training Run

Goal: Train the chosen architecture from Phase 3 on the full training set.

4a. Training

- Full training set (80% of extracted records, per temporal split)
- Validate every epoch on validation set
- Save best checkpoint by validation loss

- **Gradient clipping:** Apply gradient clipping (max norm 1.0) as a default for all training runs. This prevents occasional exploding-gradient training collapses when the model is confident but wrong (predicted 0.95, label is 0.0), which is common with BCE loss and noisy binary labels. Never harmful, cheap insurance.
- **Stochastic Weight Averaging (SWA):** Maintain an SWA model alongside the standard checkpoint — average weights from the last ~20% of epochs. SWA often produces smoother, more robust models with noisy labels. PyTorch provides `torch.optim.swa_utils`. Start SWA collection relatively late — starting too early while loss is still rapidly descending washes out the weights. Compare SWA model against best single checkpoint in Phase 5.
- Early stopping with patience ~10 epochs

Rating-based sample weighting: Rather than a hard rating floor, weight each sample by the combined rating of both players. This lets the 2200-rated games dominate gradient updates while still retaining all 102,697 games. Suggested formula:

```
weight = ((p0_rating + p1_rating) / 4000) ** 2
# 2000+2000 -> weight 1.00; 1800+1800 -> weight 0.81; 1500+1500 -> weight 0.56
```

This is strictly better than a hard cutoff — keeps all data but lets high-rated games dominate. Pass as `sample_weight` to the loss function. Compare vs. unweighted baseline in Phase 3. **Note:** With 98.8% of games having combined rating ≥ 3200 (weight ≥ 0.64), the effect is mild — the natural distribution already heavily favors high-rated games. The ablation may show negligible difference, in which case drop the weighting for simplicity.

4b. Regularisation

With ~3.1M raw records (potentially ~4.5-5M if turn-gated mirroring is adopted from Phase 3 ablation) vs Churchill's 15M, overfitting risk is higher. Apply:

- Dropout: 0.1-0.3 on hidden layers
- Weight decay: L2 regularisation ($1e-4$ to $1e-5$)
- **Label smoothing:** Apply only if Phase 3 ablation selects Strategy A (hard binary labels). If Strategy B, C, or D is selected, label smoothing is redundant — all three already soften targets. Applying both risks over-smoothing, especially for late-game positions where hard outcomes are the correct signal.
- **Mixup augmentation (optional, low priority):** Interpolate pairs of training records and their labels to create synthetic training examples. **Caveat:** All three reviewers were skeptical of mixup for discrete game states. A convex combination of two Prismata states (e.g., 2.3 Tarsiers, 0.6 Walls) is not a legal game state, and unit counts interact combinatorially rather than smoothly. Mixup sometimes helps as pure regularization despite theoretical objections, but it should be tested only after dropout, weight decay, SWA, and per-game sampling are established. If the Phase 3 ablation shows it hurts or is neutral, drop it.

```
lam = np.random.beta(0.2, 0.2)
x_mix = lam * x1 + (1 - lam) * x2
y_mix = lam * y1 + (1 - lam) * y2
```

This is a cheap, well-supported regularization technique for tabular data. Apply on-the-fly in the data loader (zero storage cost). Use a low alpha (0.1-0.2) to keep interpolated states plausible. Compare with and without in Phase 3.

Specific values should be set based on Phase 3 observations.

4c. Learning Rate Schedule

Churchill used a fixed Adam LR of $1e-5$. Cosine decay or step decay may improve convergence — worth a quick comparison during Phase 3 before locking in.

Learning rate warmup: Use a short linear warmup (500-2000 steps) before reaching peak LR. This prevents early training instability with Adam + BCE on noisy labels, especially if gradient clipping is relaxed. Standard practice, cheap insurance.

4d. Infrastructure

- Local: Intel Arc B580 (12GB VRAM, XPU backend)
- Cloud: AWS g4dn.xlarge or GCP g2-standard-4 if local is insufficient
- Monitor training vs validation curves throughout; any run where training accuracy significantly exceeds validation accuracy early should be investigated before completing

4e. Export and C++ Integration

- Export weights to binary format compatible with `NeuralNet.cpp`
- Verify all expected tensors are present (26 tensors required by C++ loader: `input_proj` + $N \times (\text{linear1}, \text{norm1}, \text{linear2}, \text{norm2}) + \text{policy}(2) + \text{value}(2)$)
- Smoke test: load weights in C++, evaluate a known state, confirm output is in expected range
- **Cross-language verification:** Evaluate a **canonical set of ~10 test states** in Python and in C++. Include: an opening state (turn 1), a mid-game state with many units, a late-game state near lethal, an empty board, a state with units under construction, a state with exhausted abilities, and mirror pairs. Store these as committed fixtures (`training/test_fixtures/`) so they can be re-run after any C++ or Python change. Confirm outputs match to 4+ decimal places. **Cast PyTorch model and dummy inputs to torch.float32 before export** — C++ uses 32-bit `float` throughout (`NeuralNet.cpp`), while Python/PyTorch defaults to 64-bit. Precision drift from float64→float32 truncation will cause spurious mismatches in the 4-decimal verification.
- **Export normalization constants** alongside weights in `schema_v1.json`. The C++ inference engine hardcodes normalization caps — verify they match the extraction pipeline's values.
- **Eval score calibration for alpha-beta:** The C++ search framework uses different score scales for different evaluation methods. Pure NeuralNet evaluation (`Eval.cpp:77-78`) scales by $\times 100$ to $[-100, +100]$. Playout evaluation returns $[-10000, +10000]$. The blend mode (`NeuralNetPlusPlayout, Eval.cpp:226-239`) correctly scales NN output by $\times 10000$ to match Playout range. **Note:** The $\times 100$ range preserves full `double` precision (no integer truncation occurs — alpha-beta comparisons use `double` throughout), so coarse granularity is NOT a concern. However, the different ranges may produce different search dynamics due to aspiration window sizes. **Action required:** When deploying the trained model, test both pure NeuralNet mode ($\times 100$ scale) and NeuralNetPlusPlayout blend mode.

If using Path A (logit/probability training), update `NeuralNet.cpp` to apply $2 * \text{sigmoid}(z) - 1$ instead of $\tanh(z)$ — mathematically different for the same raw output. Document the chosen mapping.

4f. Tactical Regression Suite

Create a set of 20-50 known positions with expected qualitative behavior. Store as committed fixtures alongside the cross-language test states. Use for fast regression testing after any change to extraction, export, C++ integration, or evaluation scaling.

Categories:

- **Forced lethal:** Active player can win this turn (value should be ~ 1.0)
- **Obvious safe defense:** Active player has clear defensive assignment (value should not drop)
- **Clearly dominant economy buy:** One buy is clearly better (policy should rank it first)
- **Mirror-symmetric states:** $\text{value}(\text{state}) + \text{value}(\text{mirror}(\text{state})) \sim 1.0$
- **Free-resource probes:** Adding significant free resources to active player should not reduce predicted value (value monotonicity)
- **Empty/degenerate boards:** Edge cases that shouldn't crash or produce NaN

This gives a fast (~ 2 minute) sanity check that catches integration bugs before expensive tournament runs.

4g. Reproducibility Checklist

For every training run, log:

- Schema version hash (from `schema_v1.json`)
- Code commit hash
- Train/val/test split manifest (replay codes per split)
- Random seed
- Opening-book setting (active/inactive, which book version)
- Search parameters (think time, player config)
- Normalization constants (must match schema)
- Model export checksum
- Label strategy (A/B/C/D) and parameters
- Sampling/weighting strategy and parameters

Store in a run-level metadata file alongside the model checkpoint. This enables reproducing any result and diagnosing regressions between runs.

Phase 5: Evaluation

Goal: Measure trained model strength against MasterBot (the Steam client's native AI, accessed via the SteamAI wrapper) and assess whether success criteria are met.

5a. Tournament Setup

- Primary opponent: **MasterBot** (Steam) via SteamAI.
- Games: **minimum 1,024** for the primary evaluation. At 1,024 games, a 3 percentage point win rate difference is detectable at 95% confidence. If the result is borderline (e.g., 52% win rate, CI barely

excludes 50%), run an additional 512-1,024 games to confirm.

- Card sets: random card sets matching normal play conditions. **Use paired matches:** for each card set, play two games with swapped starting players. This halves variance from card-set and first-player effects, making win-rate differences more detectable at the same game count.
- Think time: **3 seconds per turn** (consistent with Phase 0b benchmark). **Also report results at 1s and 7s think times** — if the neural player's advantage increases with think time, the evaluator is good but needs search depth to express it. If it decreases, the evaluator may be brittle under deeper search.
- Report Wilson confidence interval, not just point estimate. **Use paired statistical tests** (McNemar's test or sign test on pair outcomes) rather than simple binomial, since matches are paired by card set.
- **Turn-bucketed evaluation:** Report validation loss and prediction accuracy by turn quartile (opening: turns 1-8, mid-game: turns 9-18, late-game: turns 19+). A model that looks fine overall may fail badly in early or late game.
- **Card-set stratified evaluation:** Report tournament win rate separately for: (a) card sets that appear in training data vs. not seen, (b) card sets containing low-frequency units (flagged in Phase 0d), (c) exact 8-unit sets seen in training vs. unseen sets but seen units vs. rare-unit-heavy sets.
- **Every primary tournament result must note whether the opening book was active.** Run at least one evaluation with and one without.

5b. Success Criteria

- **Pass:** Win rate vs MasterBot above 50% with 95% confidence (lower bound of Wilson CI > 0.50)
- **Partial pass:** Above 50% but not yet statistically significant — run more games
- **Fail:** Below 50% — investigate data pipeline and Phase 3 findings before concluding model is fundamentally too weak
- **Note:** If the model loses to MasterBot but the margin is small (e.g., 45-48% win rate), that's a partial success indicating the approach works but needs more data or self-play iteration.

5b-2. Offline Search-Position Sanity Check (Before Tournaments)

Before running expensive tournament games, verify the model behaves sanely on positions generated by AI search (not just human replay positions):

- Run HPS + new eval on a batch of ~1,000 self-generated positions from search
- Check value consistency: positions that are objectively similar should have similar values
- Check mirror symmetry: `value(state) + value(mirror(state))` ≈ 1.0
- Check tactical sanity: does the model correctly prefer obviously winning positions?
- **Value monotonicity probes:** Verify that adding significant free resources to the active player doesn't dramatically reduce the predicted value (a small drop is fine due to edge cases, but a large swing like 0.7→0.2 indicates a feature bug). Similarly, removing opponent blockers shouldn't dramatically reduce value. These are soft sanity checks, not strict invariants.
- If the model produces erratic values on search-generated positions while performing well on validation, this signals distribution shift — the primary supervised learning limitation

This is a fast diagnostic (~10 minutes) that can save hours of inconclusive tournament games.

5b-3. Search Integration Ablation

The C++ engine supports multiple evaluation modes. Compare tournament results across:

1. **Pure NeuralNet** — neural evaluation only (`EvaluationMethods::NeuralNet`)
2. **NeuralNetPlusPlayout** — weighted blend of neural + playout evaluation
3. **Pure Playout** — baseline (existing `OriginalHardestAI` behavior)

The blend mode may outperform pure neural evaluation, especially if the neural model has weak spots on certain position types. The blend weight is configurable in the AI parameters JSON. Test at least 3 blend weights (e.g., 0.3, 0.5, 0.7 neural weight).

5c. Secondary Evaluations

- Run model vs itself (expected ~50%, confirms basic sanity)
- Run vs LiveHardestAI as a stable C++ reference for future comparability
- If Prismata players are available, qualitative play sessions against the model are valuable for assessing strategic coherence that statistics may miss
- **Value prediction accuracy on held-out test set:** For test set positions, how well does the model's predicted win probability match the actual game outcome? Report calibration plots (predicted probability vs actual win rate, binned by decile).
- **Move agreement rate (if policy head is included):** How often does the neural-guided AI choose the same move as expert human players in the same position? This measures policy alignment independent of search.
- **Value symmetry sanity check:** For a sample of test positions, verify `value(state) + value(mirror(state)) ≈ 1.0`. If this fails, there is a bug in features or mirroring.
- **Pairwise ranking accuracy:** For pairs of positions from the same game (one from the eventual winner's perspective, one from the loser's), measure how often the model assigns a higher value to the winner's position. Also measure ordering consistency across consecutive turns — does the model's value track smoothly with game progress? This may correlate with search utility better than BCE loss alone, since search relies on relative ordering between positions, not absolute calibration.

5d. Decision Gate: Proceed or Iterate?

- **Success:** Proceed to Phase 6 (self-play iteration)
- **Failure:** Diagnose root cause. Return to Phase 3 (architecture) if model architecture is suspected; Phase 1 (feature schema) if features seem inadequate; Phase 2 (data quality) if the pipeline seems suspect.
- **Partial success:** Begin self-play data generation with current model and assess whether one iteration of self-play improves it before fully rearchitecting.

Opening Book

Current State

The HPS search engine uses an opening book to bypass search for the first few turns, playing pre-computed strong sequences instead. The current book has two tiers, both extracted from the live game's AI parameters (`Prismata.swf`):

- **LiveOpeningBook** — 4 general entries: default sequences for when no specific unit triggers apply (e.g., "if Vivid Drone is in the set, open with X")

- **LiveOpeningBook2** — 50 state-specific entries covering 9 random-pool units deemed to have a high influence on opening turns (Wild Drone, Doomed Wall, Energy Matrix, Vivid Drone, Galvani Drone, Centurion, Doomed Drone, Xeno Guardian, Defense Grid). Multiple entries per unit handle different board states (P1 vs P2 starting positions, combinatorial pairings).

These were hand-coded by Lunarch Studios and reflect the meta-knowledge of professional game designers. Configuration: `bin/asset/config/config.txt` under `LiveOpeningBook` and `LiveOpeningBook2`.

Opportunity: Data-Driven Opening Book from Expert Replays

The 102,697 expert replays provide a much larger and more empirically grounded source for opening book extraction. For each card set, we can observe what 2000+ rated players actually do on turns 1-3. The existing `training/opening_book.py` was written for a previous attempt at this and may be adaptable.

Approach:

- Group replays by which random units are present in the card set
- For each unit (or combination of units), extract the turn-1 and turn-2 buy decisions from games in the top rating bracket (2000+)
- Find the most common buy sequence(s) for each triggering unit
- Generate opening book entries in the format already used by `config.txt`

Why this is valuable:

- Covers all 105 random-pool units, not just the 50 in the current hand-coded book
- Reflects actual high-level play rather than designer intuition
- Can be extracted independently of the neural training pipeline — a pure combinatorial analysis of opening data
- Strong openings reduce the importance of the early-game evaluation function, where the noisy label problem (Phase 1b) is worst

Risks and considerations:

- Opening book data should be extracted from the **training set only** — not from validation or test games. The validation/test split must be applied before extraction begins.
- An opening book biases the AI toward specific positions early in the game. If those positions are exactly the ones represented in training data, this could cause overfitting to the opening without improving evaluation quality more broadly. Monitor whether tournament performance depends on opening-book-covered card sets.
- Some card sets may have no clear consensus opening (multiple good lines exist). In those cases, it may be better to omit a book entry and let search find the move rather than forcing a suboptimal commitment.

Relationship to neural training: The opening book and the neural evaluation function are independent. Either can be improved without changing the other. However, a strong opening book effectively reduces the turns that the neural evaluator must handle well, which may make early-game label noise less consequential in practice.

Recommended action: Extract a data-driven opening book as a standalone Phase 0 task (before Phase 5), independent of the neural training schedule. **Every primary tournament result in Phase 5 should report**

whether the opening book was active. Run at least one evaluation with and one without the expanded book. If the win rate difference is large ($>5pp$), the model may be partially dependent on the book to funnel play into familiar positions rather than evaluating well generally.

Phase 6: Self-Play Iteration (Future Phase — Outline Only)

Documented here so the supervised training plan can be evaluated for compatibility with the self-play phase that follows.

6a. Self-Play Data Generation

- Use trained model as evaluation function in the JS engine matchup runner
- Generate a target of at least 100,000 self-play games (roughly matching human dataset size) before retraining
- Store in same schema format as human replay training data for compatibility

6b. Iterative Training

- Retrain on a mix of human replay data + self-play data
- Maintain human replay data in the mix throughout (never below 20% human data, following AlphaZero practice)
- Evaluate each iteration against its predecessor and against MasterBot

6c. Gumbel AlphaZero Consideration

The Gumbel AlphaZero paper (Danihelka et al., ICLR 2022) proposes improved exploration during self-play by sampling actions without replacement, providing policy improvement guarantees even with few simulations. This is directly applicable to the HPS search framework and should be evaluated for self-play iteration 2+.

Known Failure Modes

Document known failure modes and their detection/mitigation strategies. If any of these occur during training, consult this section before debugging blindly.

Failure Mode	Detection	Mitigation
Mode collapse — model outputs constant ~0.5 for all positions	Validation loss plateaus immediately; prediction histogram shows narrow spike	Check label distribution, verify gradient flow, check for normalization bug
Overfitting to early game — model performs well on turn 1-10 but poorly on turn 20+	Turn-bucketed validation loss (Phase 5a) shows diverging performance by game phase	Re-weight training samples toward late game, or use temporal sample weighting (Strategy B)

Failure Mode	Detection	Mitigation
Card set memorization — model learns unit-specific patterns but doesn't generalize to unseen card combinations	Card-set stratified evaluation (Phase 5a) shows large gap between seen and unseen sets	Consider property-based encoding (Option 4) in future iteration; add more diverse self-play data
Calibration collapse — predictions are accurate on average but badly calibrated (always ~0.52 or always >0.8)	Calibration plots show poor alignment; Brier score diverges from log-loss	Add calibration term to loss; use temperature scaling post-hoc; verify BCE loss is active
Distribution shift failure — model evaluates human-like positions well but produces unreliable values for AI search positions	Tournament win rate is low despite good validation metrics	This is the fundamental supervised learning limitation. Proceed to Phase 6 (self-play).
Feature extraction bug — one or more features are consistently zero, wrong, or misaligned	Per-feature mean/std across dataset shows zero variance, implausible ranges (negative counts, values >1.0 for normalized features), or unexpected distributions (e.g., blocking count always 0)	Compute per-feature statistics during Phase 2d validation. Compare JS extraction output vs C++ featurization on same state (3-way parity test). Check unit index mapping and normalization constants.

Resolved Design Questions

- Feature richness:** Option 2 (type-count + instance flags + supply + card-set indicator) is the starting point. Option 1 (Churchill-exact) runs as a Phase 3a baseline.
- Rating floor:** Keep the 1500 floor with rating-based sample weighting (Phase 4a). V7 data shows only 0.2% of games (~230) fall in the 1500-1599 bracket — raising the floor gains almost nothing and loses real data.
- Card set filtering:** Exclude base-only games. Base-only Prismata is a strategically distinct subgame with no relevance to competitive play.
- Value target:** Binary win outcome with soft-label modification (Phase 1b). Win prediction is correct — soft-labelling fixes the noise issue.
- Architecture prior:** Residual MLP blocks allow going deeper without degradation and are trivially portable to C++. Plain MLP is fine at 2-3 layers; residual preferred at 4+.

6. **Benchmark calibration:** MasterBot (Steam) is the primary benchmark, accessed via SteamAI. Beating a fixed AI is necessary but not sufficient — supplement with held-out prediction accuracy and move agreement metrics (Phase 5c).

What This Plan Does Not Cover

- Specific hyperparameter grid search methodology (to be detailed after architecture is selected in Phase 3)
- Cloud compute provisioning specifics (to be planned once data scale is confirmed in Phase 0)
- C++ NeuralNet integration changes (existing infrastructure reused; only weight file format must remain compatible)
- Multi-GPU training (not needed at this data scale)
- Policy learning beyond move ordering in search

Deferred Enhancements

The following items were suggested during meta-review and have merit but are deferred to iteration 2:

- **Pairwise/ranking auxiliary loss** — within same game, winning-player states should rank above losing-player states. Improves search ordering. Medium effort; consider after initial tournament results.

Optional Enhancements — Decisions

Items 5, 6, 7, 8, 11 adopted and integrated into the plan. Items 1, 2, 3, 4, 9, 10 declined.

#	Enhancement	Status
1	Player-heldout diagnostic split	Declined
2	Card-set holdout test	Declined
3	Recency weighting ablation	Declined
4	Curriculum learning on game phase	Declined
5	Feature importance analysis (SHAP/permutation)	Adopted → Phase 3g
6	Draw rate measurement	Adopted → Phase 0d
7	Pairwise ranking metric	Adopted → Phase 5c
8	SteamAI determinism check	Adopted → Phase 0b
9	Side-to-move canonicalization ablation	Declined
10	Softer rating weighting alternatives	Declined
11	Report loss-at-epoch and loss-at-time	Adopted → Phase 3c

References

- Churchill, D. & Buro, M. (2015). *Hierarchical Portfolio Search: Prismata's Robust AI Architecture for Games with Large Search Spaces*. AIIDE 2015.
- Churchill, D. (2019). *Machine Learning State Evaluation in Prismata*. AIIDE Workshop 2019.
- Danihelka, I. et al. (2022). *Policy Improvement by Planning with Gumbel*. ICLR 2022.
- Hubert, T. et al. (2021). *Learning and Planning in Complex Action Spaces (Sampled MuZero)*. ICML 2021.
- Landers, M. et al. (2025). *SAINT: Attention-Based Policies for Discrete Combinatorial Action Spaces*.
- Perolat, J. et al. (2022). *Mastering the Game of Stratego with Model-Free Multiagent Reinforcement Learning*. Science, 378(6623). (Referenced in plan overview as the closest analog for game complexity and budget constraints.)